Parking Management System

Software Architecture Design

SAD Version 2.0

Team T09
24 February 2026

Roxanne Krause (manager)
Eliud Garcia Reza
Ayman Hassen
Gabriel Ramirez

CS 460 Software Engineering

Table of Contents

Introduction 2

Design Overview 3

Component Specifications 6

Sample Use Case 13

Design Constraints 16

Definition of Terms 18

References 20

1 Introduction

This document provides an overarching view of the specific architecture that will be necessary for the implementation of the Parking Management System (PMS). The PMS will control the systems used to manage the tracking of parking within the parking structure, which will accommodate the drivers who will use the structure, and this will be facilitated by the quality of the software design. The optimization of different types of parking spots, the backup power supply that will power the PMS in the event of a power outage, and the creation of the software design are essential to the parking structure's operation. The software will consistently manage the tracking of drivers' vehicles within their respective parking spots, as well as an estimation of how many vehicles are in transit within the parking structure. The software architecture design describes a system that has the goal of providing drivers who enter the parking structure with adequate spaces to park their cars. This document will go into detail about the abstractions and implementation needed to ensure the PMS is operational as intended while meeting the regulatory requirements.

The document contains six sections, each taking on a different task necessary for a sufficient software architectural design. Section 2 goes into the details about the design of the system through the illustration of the design structure depicted through a diagram of the system architecture. Section 3 will describe the various necessary components that are represented in the system diagram, describing their interaction within the system as well as their software interface methods. Section 4 covers the various use cases of the PMS and describes some of the important scenarios a driver will encounter through addressing the necessary software logic. Section 5 outlines several design constraints that have been taken into account when designing the PMS. Section 6 contains the definitions of technical terms that are essential to understanding the PMS and are used throughout the document. All of the materials used as references are listed at the end of the document.

2 Design Overview

In this section, the architectural design of all components needed for software logical control of the PMS will be summarized and visualized through UML diagrams. The main software components can be split into 2 categories: PMS and administrative.

2.1 PMS Software Components

In *Figure 1*, an overview of the software architecture of the PMS is visualized. Included are connections between the objects contained within the PMS software, as well as the data streams consistently communicated by external interfaces. The Administrator process will be overviewed in Section 2.2.

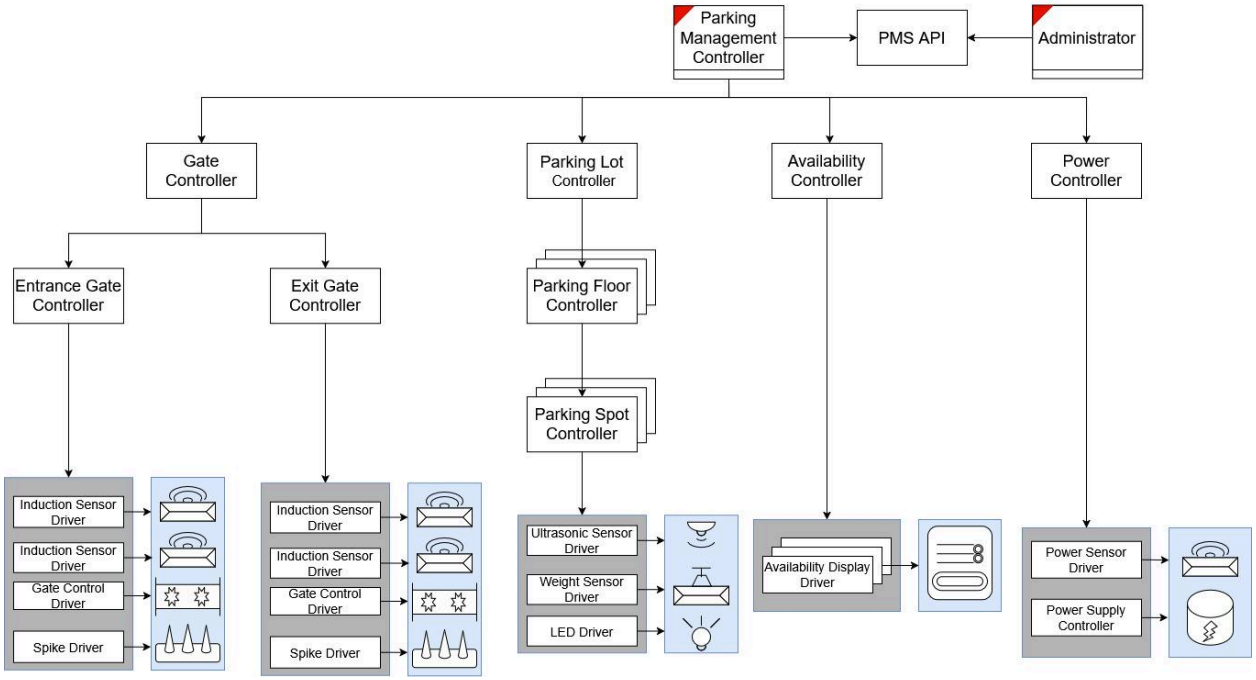

Figure 1: PMS Software Architecture Overview

With all of these components working together, the following is an explanation of each component. Section 3 will elaborate on each of these components' actions on a technical level.

Parking Management Controller (PMC)
The Parking Management Controller is the main program and root of all interactions of the software system. This controller acts as a delegator of tasks to all other software components based on the constant influx of signals from external interfaces and administrative requests.

Gate Controller
The Gate Controller issues directives to the exit and entrance gate controllers. As directed by the PMC, it will manage state changes and issue directives to the two gates reactionarily based on external interface signals and map specific sensors to the pertinent gate.

Exit/Entrance Gate Controllers
These components will offer custom state control and actions based on the type of gate (exit or entrance). This allows for flexibility in states between the two gates since they do not share every state. The main external interfaces that transmit and receive communication are the two induction sensors, the gate mechanical control, and the spike mechanical control.

Parking Lot Controller
The Parking Lot Controller holds state and funnel directives for the overarching parking lot, specifically including the major components of individual floors and spots. As requested by the PMC, it will delegate all state changes and issue directives to the floor controllers as is appropriate, which likewise take action on individual spots as needed.

Parking Floor Controller
This component acts as a logical controller of a specific floor of the parking lot. This allows for fine-tuned control over an individual floor's availability state as required by PMS to track. All parking spots are then under the control of the individual floor, thus this acts as a controller of the state and directives for all parking spots under its purview as well.

Parking Spot Controller
This component acts as a logical controller of a specific parking spot within a floor of the parking lot. This will allow for state control at the specific spot based on the external interface signals that are used to determine these state changes. The devices that will have signals that require updates from the parking spot controller are their two associated sensors, with the LED accepting a signal to visually indicate the state. Any further output signals to configure the device drivers and attached hardware will be delegated through this controller.

Availability Controller
The Availability Controller tracks all availability counts. Based upon signals from external interfaces, the PMC will issue directives to other components to manage state and actions needed, but this component specifically takes directives to update central internal tracking numbers that are then output to the external availability displays.

Power Controller
This object is a logical controller of the actions needed if the power sensor detects a power outage from the main power supply. This will be able to take action to command a power source switch given this condition.

Device Drivers
As seen in *Figure 1*, the device drivers are the translation layer between primary software components of the PMS and the hardware needed to run the system. Input from these drivers will funnel through the top-level PMC, and all reactionary logic to these signals will be delegated as seen to lower-level components, whether just as internal tracking updates or output instructions to the hardware through the drivers.

2.2 Administrative Software Components

In *Figure 2*, a brief overview of the simple admin application architecture is visualized. The application requires a simple request/response communication with the primary PMS software, as seen in *Figure 1*. The following architecture accomplishes these goals while keeping the design straightforward.

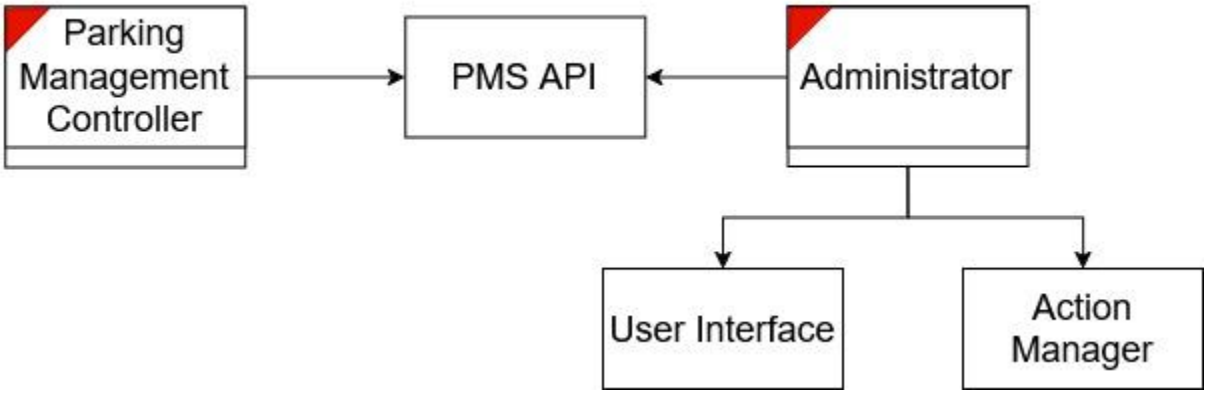

Figure 2: Admin Application Software Architecture Overview

Administrator
The Administrator is a separate process representing an administrative application connection to the PMS if it exists. This relies upon a communication thread between the PMS software and the administrator application that allows for override actions to be communicated through the PMC, and directives will be issued accordingly, enabling intercommunication.

PMS API
A shared software interface component among the administrator application and the PMC to momentarily store and help relay requests from an administrator process to the PMS. This allows the PMC to output information regarding the status of administrator requests and general information requested regarding device and data-related information.

User Interface
The User Interface component of the administrative application will control all visual formatting of the application itself, focused only on setting up the look and user experience of the front-end application.

Action Manager
The Action Manager complements the User Interface: this will define all of the actions attached to interactables, such as buttons, forms, and so on, defined by the User Interface. This component will package all data needed or given with an action taken by the administrator into serializable information to transmit to the PMC via the PMS API.

3 Component Specifications

This section will, in further detail, specify the technical details in how the PMS software will receive signals from external hardware and consequent of handling this data. Starting from the main PMC program root receiving external signals, state modifications and directives are handed down to lower-level components to carry out internal tasks and make changes to the hardware state as necessary through drivers. This elaborates on the tasks that the PMC will need from every component to ensure tracking flow. All functions mentioned without a return type specified will default to void.

3.1 PMS Components

The following components are objects detailed in the diagram of Section 2.1. Here, we elaborate on the accessible information and action end-points necessary for each component to enable the PMC's logical control based on the reception of external signals. Each component is handed the required information for the pertinent directive function down through the hierarchy in *Figure 1*. This will be primarily maintained through ID mapping.

Parking Management Controller

The PMC acts as a funnel of communication from signals from all the sensors within the system. Its primary role is to receive a signal and pass it in an expected way to the next, lower-level controller component. One unique action it takes responsibility for is closing and opening the parking lot in response to an admin request, which requires special actions through multiple separated components.

- ***closeParkingLot():*** Given an administrator request, initiate changing the entrance gate state to LOCKED, updating availability displays' counts to 0, and changing availability displays' log message to "FULL".
- ***openParkingLot()***: Given an administrator request, initiate changing the entrance gate state to UNLOCKED, updating availability displays' counts to current counts, and clearing availability displays' log messages.

Gate Controller

The Gate Controller acts as a funnel for gate-specific requests. Specifically, it takes directives from the PMC to conduct gate state changes based on gate ID, such as initiating the opening and closing procedures of both gates. Additionally, it will initiate the locking and unlocking procedures of the entrance gate only.

- ***lockEntranceGate()***: Given an administrator request or no availability left in the lot, initiate a gate locking procedure when possible.
- ***unlockEntranceGate()***: Given an administrator request or no availability left in the lot, initiate a gate unlocking procedure when possible.
- ***moveGate(GateId gate, bool open):*** Given the entrance or exit gate ID, directive to open (*true*) or close (*false*) the gate.

Exit Gate Controller
The Exit Gate Controller, as directed by the Gate Controller, will solely control the open and close output signals to the gate drivers associated with the exit gate mechanical control and directional spikes. It will also maintain the current state of the gate as a tracking mechanism. The possible states are OPEN or CLOSED.

- ***openGate()***: Directive to change the exit gate to OPEN state.
- ***closeGate()***: Directive to change the exit gate to CLOSED state.

Entrance Gate Controller
The Entrance Gate Controller, as directed by the Gate Controller, will solely control the open and close output signals to the gate drivers associated with the exit gate mechanical control and directional spikes. It will also maintain the current state of the gate as a tracking mechanism. The possible states are OPEN, CLOSED, or LOCKED.

- ***lockGate()*:** Directive to change to LOCKED state if in CLOSED state.
- ***unlockGate()***: Directive to change to CLOSED state if in LOCKED state.
- ***openGate()***: Directive to change the entrance gate to OPEN state.
- ***closeGate()***: Directive to change the entrance gate to CLOSED state.

Parking Lot Controller
The Parking Lot Controller will maintain specific states of the entire lot. The possible states are OPEN or FULL. All directives based on signals to the PMC specifically for parking tracking will be passed to this to check for action viability and initiate actions if possible. Further, this will maintain an internal ease-of-access tracker of total lot availability.

- ***int getCurrentAvailableCount(SpotType type):*** Return the number of spots AVAILABLE currently in the entire lot for a specific parking spot category.

Parking Floor Controller
The Parking Floor Controller will maintain specific states of a specific floor. The possible states are OPEN or FULL. All directives to change the state of a specific spot will funnel through its associate floor controller. Further, this will maintain an internal ease-of-access tracker of floor availability.

- ***int getCurrentAvailableCount(SpotType type):*** Return the number of spots AVAILABLE currently on this floor for a specific parking spot category.

Parking Spot Controller
The Parking SpotController will maintain specific states of a specific parking spot. The possible states are AVAILABLE, UNAVAILABLE, or OCCUPIED. All directives to change the state of a specific spot will be received by this, and the needed tracking updates or output actions to drivers will be controlled by this component.

- ***updateSpotAvailability(SpotId spot, bool vehicleDetected):*** Update any internal tracking of sensors on a trigger. Updates parking spot availability state to OCCUPIED or AVAILABLE based on the vehicleDetected for both sensors. The sensor ID will be encoded within the spot ID, allowing for ease of transmission.
- ***forceAvailable():*** Given an administrator request, change the spot state from UNAVAILABLE to AVAILABLE if possible.
- ***forceUnavailable():*** Given an administrator request, change the spot state from AVAILABLE to UNAVAILABLE if possible.
- ***changeSpotType(SpotType type):*** Given an administrator request, change the type of this spot (for example: NORMAL -> EV).
- ***changeLEDDefault(Color color):*** Given an administrator request, change a parking spot's LED default availability color to the corresponding RGB values.
- ***configureSensor(SensorID sensor, SensorUpdate update):*** Given an administrator request, change a sensor's range and/or sensitivity for the spot. A data package (wrapped as a SensorUpdate object) for a specific SensorId is required, and will be translated into a digital signal to transmit to the corresponding sensor driver.

Availability Controller
The Availability Controller acts as a central availability tracking component that holds all current counts of availability in a central component. It will take updates as directed by the PMC, and then output signals to the availability display drivers to update the display feedback.

- ***updateAvailabilityTracking(SpotType type, int floor, Action incOrDecrement):*** Access point to update an internal tracking number held by the availability controller. In turn, the controller will output a digital signal to communicate the availability numbers of a specific category to the availability displays.
- ***updateLogMessage(String msg):*** Given both automatic and administrative actions, update the log message on all availability displays.
- ***closeLot():*** Update all availability displays to 0 for all counts and change the log message to "CLOSED". Maintain internal tracking.

Power Controller
The Power Controller is in sole control of issuing signals to the power supply controller, which is able to switch between main and battery power in the event of power outages. PMC will receive the signals from the power sensor and, in turn, initiate a power switch through this component.

- ***switchToBattery():*** Issue a directive, given the main power has failed, to switch to the emergency battery pack as a power source. Will transmit a digital signal to the power supply controller to change the power source.
- ***switchToMain():*** Issue a directive to switch the main power grid as a power source, given that it is available. Will transmit a digital signal to the power supply controller to change the power source.

3.2 Administrative Software Components

The following components, as visualized in *Figure 2*, are the required objects of the main administrative application. Simply, the Administrator acts as the main application execution run, and the User Interface and Action Manager work together to create a responsive and well-designed application.

Administrator
The Administrator is primarily acting as a top-level request sender, as well as being the main entry point to start the application. This will hold socket connection threads to both output requests through the PMS API and receive responses from the PMS.

- ***sendUpdate(Update updateRequest)****:* Access point to send through a parking lot state update as a serializable data stream to a connected admin application's socket. This is called upon to send through requested state changes to the PMS.

Action Manager
The Action Manager is used for defining (1) actions the admin can take and (2) packaging the necessary information for the action for sending to the PMS.

- ***Update openLot():*** Admin request to initiate lot opening through PMS.
- ***Update closeLot():*** Admin request to initiate lot closing through PMS.
- ***Update markSpotUnavailable(SpotId spot):*** Admin request to mark a specific spot as unavailable through PMS.
- ***Update markSpotAvailable(SpotId spot):*** Admin request to mark a specific spot as available through PMS.
- ***Update lockGate():*** Admin request to lock the entrance gate through PMS.
- ***Update unlockGate():*** Admin request to unlock the entrance gate through PMS.

- ***Update changeLogMessage(String msg):*** Admin request to change the log message on the availability displays through the PMS.
- ***Update changeLEDDefault(SpotId spot, Color color):*** Admin request to change the default availability color of a specific spot.
- ***Update changeSpotType(SpotType type, SpotId spot):*** Admin request to change the type of a specific spot through the PMS.
- ***Update configureSensor(SensorId sensor, SensorUpdate update):*** Admin request to adjust the sensor, with parameters wrapped in a SensorUpdate to send to the PMS.

User Interface
The User Interface is a component that controls (1) the initial visuals of the GUI and (2) updates components of the GUI as needed in response to PMS state changes.

- ***updateUI(UpdateType update)*** Directive to update the user interface according to the state update type that has been passed to the Administrator via notification.

3.3 External Interface Drivers

The following components are software translation layers that can receive digital signals over wire with certain directives from the PMS. When acting as output devices, these digital signals result in a hardware response when output signals come from the PMS. When acting as input, a digital signal will be relayed to the PMS, and the appropriate internal logic actions (as detailed in Section 3.1) are taken.

Induction, Weight, Ultrasonic Sensor Drivers
The Induction, Weight, and Ultrasonic Sensor Drivers all have very basic functionality. They can receive an enabling or disabling signal from associated controllers, and likewise fire a signal to the top-level PMC when triggered by a vehicle detection. The following methods are split into output and input signals for clarity.

Output Signals from PMS
- ***enable():*** Directive to enable a sensor, effectively turning it on.
- ***disable():*** Directive to disable a sensor, effectively turning it off.

Input Signals to PMS
- ***(ComponentId gateOrSpotId, bool vehicleDetected) trigger():*** Returns a binary indicator of a vehicle detected (1) or not (0) on change and the relevant component ID upon the detection/non-detection of a vehicle.

Gate Control Driver
The Gate Control Driver acts only to receive output signals from the associated Entrance or Exit Gate Controllers. It has the baseline functionality of emitting a mechanical response from the gates: open, close the gate, and set the LED on the gate to flashing or completely off.

- ***open()***: Directive to open the gate arm, set gate light mode to flashing on.
- ***close()***: Directive to close the gate arm, set gate light mode to flashing on.
- ***flash(bool flashingOn)***: Directive to set gate light mode to flashing on or off.

Spike Driver
The Spike Driver acts only to receive output signals from the associated Entrance or Exit Gate Controllers. It has the baseline functionality to only trigger the mechanical action of lowering or raising the directional spikes.

- ***lower()***: Directive to lower the spikes.
- ***raise()***: Directive to raise the spikes.

LED Driver
The LED Driver acts only to receive output signals from the associated Parking Spot Controller. It only has the functionality to change color as a visual response to parking spot state changes.

- ***color(Color color):*** Directive to change the color to a given encoded RGB value.

Availability Display Driver
The Availability Display Driver acts only to receive output signals from the associated Availability Controller. It will take a stream of serialized data to specify new counts for spot types and, optionally, specific floors or log messages to display on the physical availability displays throughout the lot.

- ***totalAvailabilityDisp(SpotType type, String count, int floor):*** Directive to update the total or floor availability count for a specific spot type on the availability displays.
- ***logDisp(String msg):*** Directive to update the log message on the availability displays.

Power Sensor Driver:
The Power Sensor Driver acts only to relay input signals to the PMC, signifying a lapse in power from the current power source (main or battery).

- ***(SensorId id, bool powerFailDetected) trigger():*** Returns the state of the main power source being live (1) or not (0) on change, and the sensor ID as identifying information.

4 Sample Use Case

*This section describes some typical use cases and includes a diagram that shows how different users will interact with the Parking Management System. Below, 3 Distinct Use Cases compose the majority of the interactions within the Parking Management System, including the Driver's interactions at Entry/Exit gates (**Use Case 1**) and the Parking Spots (**Use Case 2**), and the Administrative orders given to the PMC (**Use Case 3**).*

4.1 Use Case 1

Use Case 1: Vehicle Entry/Exit and Parking Structure Capacity Management
Actor: The Driver
Goal: To handle the core operational flow of vehicles entering and exiting the parking structure while maintaining accurate capacity counts at both the structure and floor levels.
Preconditions: The PMC must be operational, the entrance and exit gates are functional, and their respective controllers are initialized; the sensors at the entrance and exit are enabled, and the availability displays are powered on and connected.
Trigger: A single Vehicle approaches the entrance gate or the exit gate.
Scenario:

A. Vehicle Entry:
    1) The vehicle arrives at the entrance gate, which activates the entrance induction loop sensor. The sensor driver sends a ***trigger()*** signal to the PMC.
    2) The PMC asks the Parking Lot Controller, using ***getCurrentAvailableCount(SpotType spotType)***, to verify the available spaces.
    3) If the spaces are available, the PMC will notify the Gate Controller of a vehicle presence at the entry gate. Gate Controller then will direct the Entrance Gate Controller, if not locked, to ***openGate(),*** which signals the gate and directional spike drivers.
    4) The gate driver receives an ***open()*** signal, which raises the gate and sets the gate lights to flashing. The spike driver receives a ***raise()*** signal, which raises the spikes.

5) The vehicle passes through, and the second sensor sends a signal to PMC via ***trigger()***.
6) The PMC will notify the Gate Controller that a vehicle is clear of the entry gate. Gate Controller then will direct the Entrance Gate Controller to ***closeGate(),*** which signals the gate and directional spike drivers.
7) The gate receives a ***close()*** signal, which lowers the gate and sets the gate lights to flashing. The spike driver receives a ***lower()*** signal, which lowers the spikes.
8) The PMC updates the Availability Controller through ***updateAvailabilityTracking(SpotType type, int floor, Action incOrDecrement)*** to increment the total lot in-transit count and, in turn, refresh the availability displays.

B. Vehicle Exit:
1) The vehicle arrives at the exit gate, which triggers the exit induction loop sensor. The sensor driver sends ***trigger()***.
2) The PMC will notify the Gate Controller of a vehicle presence at the exit gate. Gate Controller then will direct the Exit Gate Controller to ***openGate()***, which signals the gate driver with ***open()*** and signals the directional spike driver with ***raise()***.
3) The vehicle leaves, and when sensor signals ***trigger(),*** the PMC will notify the Gate Controller of a vehicle being clear of the exit gate, which in turn commands the Exit Gate Controller to ***closeGate()***. This then signals the gate driver with ***close()*** and signals the directional spike driver with ***lower()***.
4) The PMC updates the Availability Controller through ***updateAvailabilityTracking(SpotType type, int floor, Action incOrDecrement)*** to decrement the total lot in-transit count and, in turn, refresh the availability displays.

4.2 Use Case 2

Use Case 2: Parking Spot Occupation and State Update
Actor: Parking Spot Sensor, The Driver
Goal: Accurately reflect the occupancy status of a parking spot and update the floor and structure availability.
Preconditions: The vehicle has already entered the parking structure, the parking spot sensor is enabled and operational, and the Parking Spot Controller is initialized with the spot's Id and its type.
Trigger: A vehicle parks in a parking spot, triggering the sensor

Scenario:

1. The vehicle parks at the parking spot, which activates the weight and ultrasonic sensor. The sensor drivers send ***trigger()*** signals to the PMC.
2. The PMC routes the signal to the Parking Lot Controller, which transfers the signal to the appropriate Parking Floor Controller. Then, the Parking Spot Controller is directed to ***updateSpotAvailability(SpotId spot, bool vehicleDetected)***. The controller then verifies both sensor signals received and transitions the spot to OCCUPIED.
3. The Parking Spot Controller updates the spot's LED through the LED Driver: ***color(Color color)***. Occupation notification is returned to the Parking Floor Controller.
4. The Parking Floor Controller decrements its internal availability counts for this spot type. The floor update counts are returned to the Parking Lot Controller. If the floor is full, change state to FULL.
5. The Parking Lot Controller decrements its total lot availability counts for this spot type. The lot availability counts are returned to the PMC. If all floors are full, change the state to FULL.
6. PMC directs the Availability Controller to ***updateAvailabilityTracking(SpotType type, int floor, Action incOrDecrement)*** for both general availability and in-transit counts.
    a. If the Parking Lot Controller is FULL, PMC sends an additional directive to the Availability Controller to ***updateLogMessage(String msg)*** with a full lot indicator message***.*** PMC also sends a directive to the Gate Controller to ***lockEntranceGate()***.
7. The Availability Controller sends commands to the Availability Display Driver: ***totalAvailabilityDisp(SpotType type, String count, int floor)*** to update all displays. If a log update is received ***logDisp(String msg).***

4.3 Use Case 3

Use Case 3: Administrative Needs for Parking Lot State Modification: Closing Lot
Actor: System Administrator
Goal: To allow administrative users to modify parking lot configuration, override states, or customize the system behavior. Specifically, the need to close the lot for maintenance
Preconditions: The administrator has access and has logged into the application, the administrative software is connected to the PMS, and the PMS is operational.
Trigger: The Administrator initiates a request to close the lot through the admin user interface
Scenario:

1. The administrator presses a button to request a lot closure in the User Interface (UI). The associated function in the Action Manager, ***closeLot(),*** tied to the request submission button, creates an Update package which adheres to the expected format of the PMS API.
2. This Update package is then sent through a communication socket by calling ***sendUpdate(Update updateRequest)*** in the Administrator component.
3. This request is then transmitted to the PMS API over a socket and unpacked for the PMC.
4. The PMC then determines the request type. For a close lot request, it will issue a ***closeParkingLot()*** directive.
5. The Parking Management Controller implements the following instructions:
    - Request the Gate Controller to ***lockEntranceGate()***.
    - Tells the Availability Controller to set all the available displays to 0 and log messages to "CLOSED" using ***closeLot()***.
6. After it successfully executes, the PMC packages a successful lot closure notification for the PMS API, which in turn packages the update to send back to the Administrator over the connecting communication socket.
7. When the Administrator receives this update, it will ***updateUI(UpdateType update)***.
8. The UI displays a confirmation message to the administrator, and the updated system state is reflected on the admin dashboard.

5 Design Constraints

The design constraints, as follows, are to be adhered to carefully in the development and consideration of the PMS software. These constraints will support what is outlined in the Requirement Definition Document while adding specificity to the constraints on software for the system.

5.1 PMS Software Constraints

- PMS software must be able to receive requests from and send responses to the administrative application (if it is running). This communication must reside through designated server ports/sockets.
- All hardware must have an associated device driver to receive digital commands from the PMS.
- Software must all be written using the programming language C++, using it for full logical and user-interface development.
- Software must prioritize administrator requests over passive updates depending on the kind of administrator override. For example:

    - The administrator locks the entrance gate; PMS will no longer automatically open the entrance gate on vehicle approach.
    - Administrator makes parking spots unavailable; PMS, until further notice, will no longer actively track the occupation of this spot.
    - Administrator makes an invalid request; PMS continues regular operations (as well as any previously given administrator requests).
- Repeated administrative requests made (i.e., make parking spots unavailable) will be ignored if the request has already been received once, and the administrator will be notified that such tasks are already ongoing.

5.2 Administrative Application Constraints

- The user interface of the administration application must be clear and easy to use. Further, a graphical display of each floor must be easily accessible and clear for oversight needs.
- The user interface must update dynamically as a result of state changes communicated by the PMS.
- There must be a username and password verification to access the security application.
- There must be a secure database used for password verification for the application.
- There must be an administrator on-site to use the application. There is no remote access permitted.
- The administrator will not compete with the software as they are deploying the action they request (based on request type).
- The updates requested by the administrator must be accepted and carried out in no more than 10 seconds.
- Updates to the GUI interface showing the state of lot components must take no longer than a 5-second delay between PMS update and visual feedback.

5.3 Implementation Directives

In order to meet the client's requirements, we must ensure the software can not only accurately track the number of available spots by type, but also be capable of continued operation, with, in the worst-case scenario, limited functionality. Additionally, administrators should be able to do their overrides with little to no frustration from the system during regular parking operations. Any inaccurate counts of parking spots may cause frustration to drivers who wish to use the parking structure, so the software must be able to track the number of available spots alongside its most recent administrative command (if any) for any given parking spot. Lastly, the software must ensure to maintain a good flow of traffic by minimizing the time between detecting drivers and opening the gates after receiving related information. This is especially true to help avoid drivers from

overcrowding a potentially full parking structure, or only one car entering/leaving the structure at a time.

6 Definition of Terms

In this section, we define some relevant terms that are used frequently throughout the paper for accurate communication. The reader can use this section as a future reference while going through the document.

Administrator - Generally, owners of the structure or specifically assigned personnel in charge of managing the PMS, enabling manual override of certain functionality, and manually updating parking space availability whenever necessary.
Administrative Application - The software application that is provided for administrators to make manual changes to the PMS. Sometimes simply referred to as an *application*.
Availability - The status of a parking spot, which will be either filled (vehicle parked) or available (no vehicle parked).
Availability Display - The required main display is made available in front of the parking lot to provide availability counts for various spot types. Additionally, it can refer to displays on each floor of the lot displaying the same information as the aforementioned main display.
C++ - A high-performance programming language that is an extension of the C programming language by adding object-oriented functional features used to build complex software.
Device Driver - A software translation layer that allows for an interface between digital signals received and instructions for the hardware it controls [1].
Driver - Individuals who drive a vehicle (of any type) and desire to use a parking space for their vehicle.
GUI - "Graphical User Interface". Primarily refers to the front-end application that the administrator will issue requests to. Sometimes referred to as the user interface.
In Transit - A status of a vehicle referring to when the vehicle is within the garage, but not parked. Additionally, the count of these is the number of vehicles with this status.
LEDs - A light-emitting diode, used for indicating parking space availability.
Override - An action from the administrator digitally that takes priority in running and prevents regular activities from the PMS and its devices pertaining to said action.
PMC - The Parking Management Controller, acting as a delegator of tasks to all other software components based on the constant influx of signals from external interfaces and administrative requests.
PMS - The Parking Management System, consisting of all the devices and software necessary to manage and oversee parking lot operations.

Port - A virtual endpoint within an operating system that acts as a specific communication doorway that can allow network traffic to be directed to the right destination.
RGB - A color system that uses red, blue, and green to blend into a wide array of other colors.
Server - A computer software system that can manage network resources and deliver functionality to other devices over a network.
Spikes (Spiked Speed Bump) - A controlled traffic-calming device used to direct traffic to one direction using spikes and a raised bump. Occasionally referred to as *directional spikes*.
Vehicle - a metallic, wheel-based machine primarily used for transporting people, including cars, trucks, motorcycles, etc.

7 References

[1] “Device Driver”, wikipedia.org, https://en.wikipedia.org/wiki/Device_driver. Accessed Feb. 9th 2026